Spring boot - Part9 (Role based Authorization - Annotations)

Role based Authorization - (Annotation)

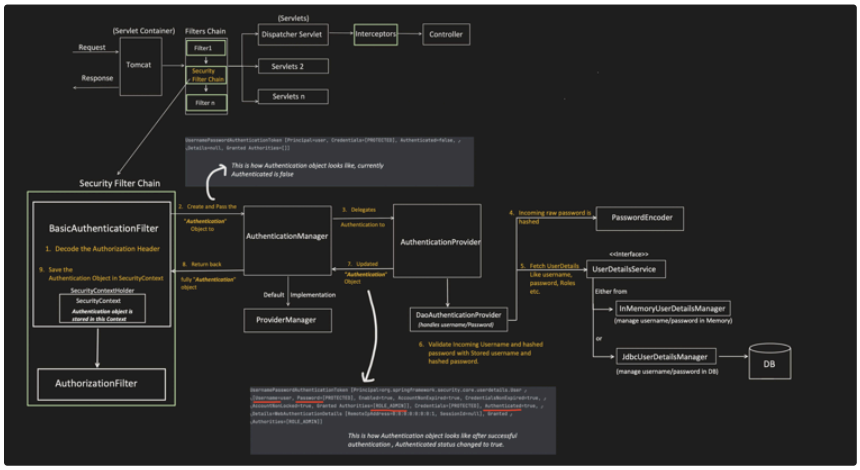
- We have already covered different type of Authentications.
- And with that, we have also covered, how at Security Filter layer itself we can do authorization.

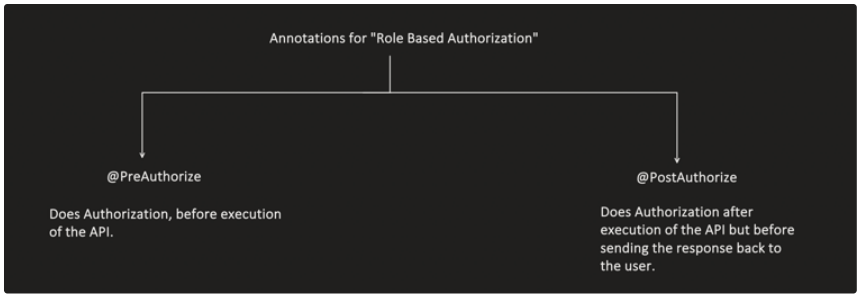
```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/users").hasRole("USER")
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults());

        return http.build();
    }
}

```

- **But in large scale applications, where we might have 100s of APIs, in that scenarios managing the roles at Security Filter might become difficult and cause scalable issue.**

Concept && Coding

That's where, Annotation based "Role Based Authorization" comes into the picture. And these annotations are used within our Controller class.





Lets create User First (Dynamic one)

We have already seen its implementation in

```java
@Entity
@Table(name = "user_login")
public class UserLoginEntity implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    private String role; // e.g., "ROLE_USER" or "ROLE_ADMIN"

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private List<UserPermissionEntity> permissions = new ArrayList<>();

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<GrantedAuthority> authorities = new HashSet<>();

        authorities.add(new SimpleGrantedAuthority(role));

        permissions.forEach(permission ->
            authorities.add(new SimpleGrantedAuthority(permission.getName()))
        );

        return authorities;
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return username;
    }
}

```

Concept && Coding

```java
@RestController
```

```java
public class UserLoginController {

    @Autowired
    UserLoginEntityService userLoginEntityService;

    @Autowired
    PasswordEncoder passwordEncoder;

    @PostMapping("/user-login")
    public ResponseEntity<String> login(
            @RequestBody UserLoginEntity userLoginEntity) {

        // Hash the password before saving
        userLoginEntity.setPassword(
                passwordEncoder.encode(userLoginEntity.getPassword())
        );

        userLoginEntityService.save(userLoginEntity);

        return ResponseEntity.ok("User registered successfully!");
    }
}

```

```java
@Entity
@Table(name = "user_permission")
public class UserPermissionEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // e.g., ORDER_READ, SALES_READ
    private String name;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

Concept && Coding

```java
@Service
public class UserLoginEntityService implements UserDetailsService {

    @Autowired
    private UserLoginEntityRepository userLoginEntityRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        return userLoginEntityRepository.findByUsername(username)
                .orElseThrow(() ->
                        new UsernameNotFoundException("user not found"));
    }

    public UserDetails save(UserLoginEntity userLoginEntity) {
        return userLoginEntityRepository.save(userLoginEntity);
    }
}

```

```java
@Repository
public interface UserLoginEntityRepository
        extends JpaRepository<UserLoginEntity, Long> {

    Optional<UserLoginEntity> findByUsername(String username);
```

```
}

```

Now, we should be able to create user, and in this one User can have 1 role like ROLE_ADMIN, ROLE_USER etc. But we can give many permissions like ORDER_READ, SALES_DELETE etc.... (more granular level permissions)

Now, User Creation part is done, lets update our Security Config file.

```
@Configuration
@EnableWebSecurity
  /*
    Without enabling this, @PreAuthorize and
  @PostAuthorize annotations will be ignored.
   */
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws Exception {

        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/user-login").permitAll()
                .anyRequest().authenticated()
        )
        .sessionManagement(session -> session

.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

Concept && Coding

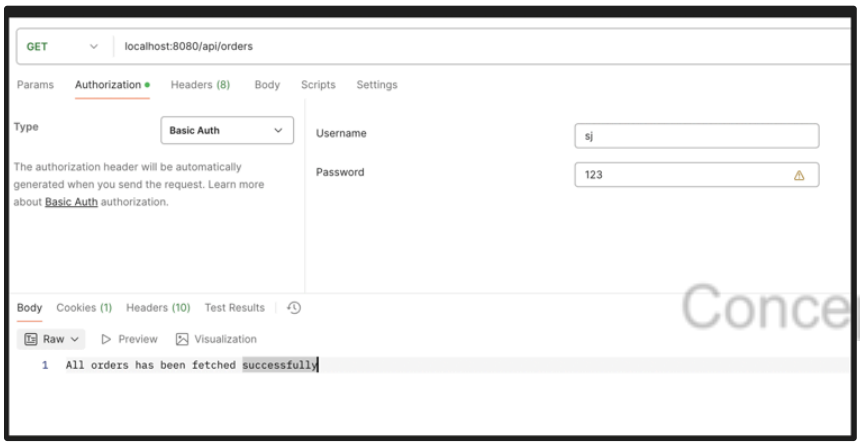
Now, lets create 2 Controller class, one for ORDER and another for SALES for testing purpose

```
@RestController
@RequestMapping("/api")
public class OrderController {

    @GetMapping("/orders")
    @PreAuthorize("hasRole('USER') and hasAuthority('ORDER_READ')")
```

```
    public ResponseEntity<String> readOrders() {
        return ResponseEntity.ok("ALL orders has been fetched successfully");
    }
}

```

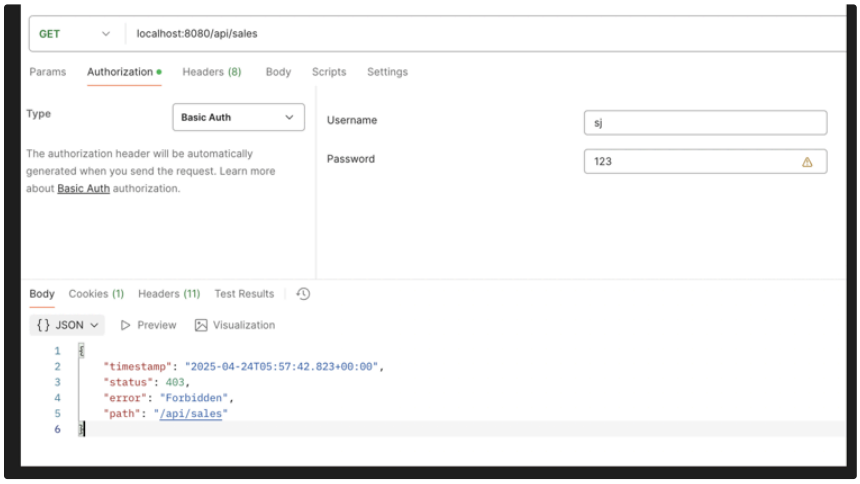
```
@RestController
@RequestMapping("/api")
public class SalesController {

    @GetMapping("/sales")
    @PreAuthorize("hasAuthority('SALES_READ')")
    public ResponseEntity<String> readSalesDetails() {
        return ResponseEntity.ok("ALL Sales details has been fetched successfully");
    }
}

```

Above we have created user, who has both the permissions ROLE_USER and ORDER_READ, so API is successfully executed.



Concept && Coding

Now, when we try to access /sales API, which above user do not have permission, request has thrown exception.




```
@PreAuthorize("hasRole('USER') and hasAuthority('ORDER_READ')")
```

Spring Boot, treat both "hasRole" and "hasAuthority" in a similar way, only difference is that :

For "hasRole" : "ROLE_" is appended

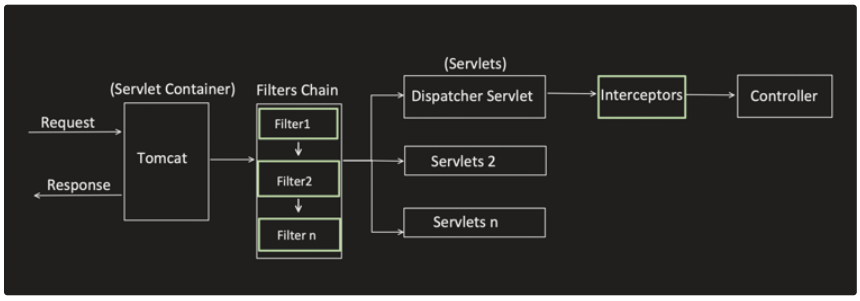
SecurityExpressionRoot.java

```java
@Override
public final boolean hasAuthority(String authority) {
    return hasAnyAuthority(authority);
}

@Override
public final boolean hasAnyAuthority(String... authorities) {
    return hasAnyAuthorityName(null, authorities);
}

@Override
public final boolean hasRole(String role) {
    return hasAnyRole(role);
}

@Override
public final boolean hasAnyRole(String... roles) {
    return hasAnyAuthorityName(this.defaultRolePrefix, roles);
}

private boolean hasAnyAuthorityName(String prefix, String... roles) {
    Set<String> roleSet = getAuthoritySet();
    for (String role : roles) {
        String defaultedRole = getRoleWithDefaultPrefix(prefix, role);
        if (roleSet.contains(defaultedRole)) {
            return true;
        }
    }
    return false;
}

```

Concept && Coding

Now, one question comes to mind is, how this Authorization methods invokes before invocation of the Controller method.

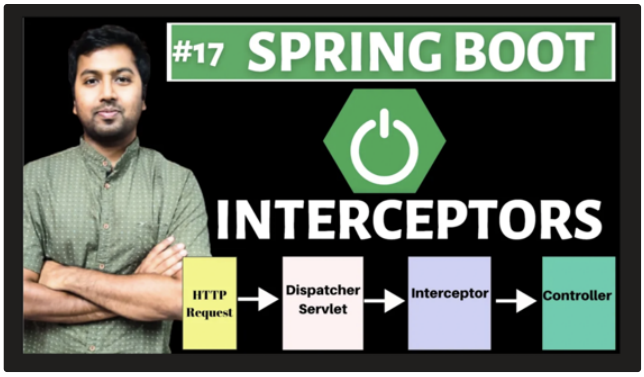
Its because of INTERCEPTORS

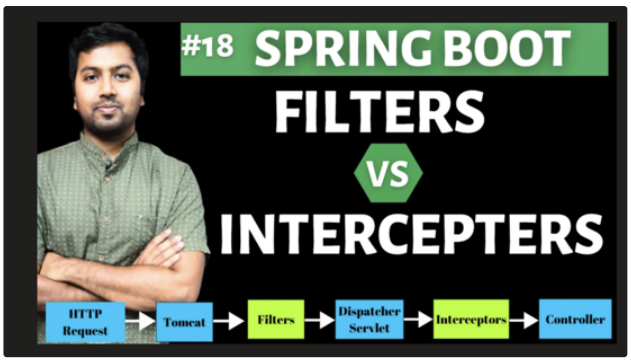


@PreAuthorize Annotation is intercepted by ***AuthorizationManagerBeforeMethodInterceptor***

What and how Interceptors works and how they are different from filters?

We have already covered both the topics in depth.






```
@PreAuthorize("hasRole('USER') and hasAuthority('ORDER_READ')")
```

This expression uses SpEL (i.e. Spring Expression Language)

Spring framework has *SpelExpressionParser* class, which help in compilation of these strings.

Concept && Coding

Below is the high level flow:

More Logical Operators

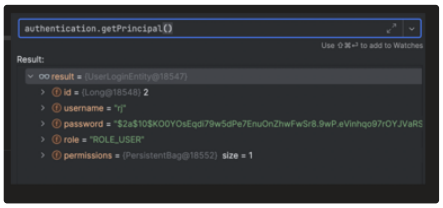
Operator	Description	Example
and	Logical AND	hasRole('ADMIN') and hasAuthority('READ')
or	Logical OR	hasRole('ADMIN') or hasRole('USER')
not	Logical NOT	not hasRole('ADMIN')
!	Also logical NOT	!hasAuthority('DELETE')

Relational Operators

Operator	Description	Example
==	Equal	#value == 15
!=	Not equal	#value != 15
<	Less than	#value < 100
>	Greater than	#value > 100
<=	Less than or equal	#value <= 15
>=	Greater than or equal	#value >= 90

Concept && Coding

```java
@PreAuthorize("#id == authentication.principal.id")
@GetMapping("/users/{id}")
public User fetchUserDetails(@PathVariable Long id) {
    return userServiceObject.fetchUserDetails(id);
}

```

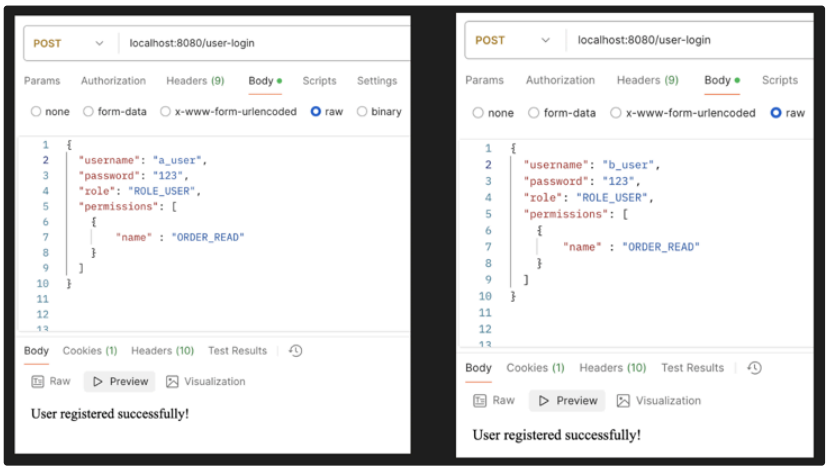


@PostAuthorize

- Does Authorization after execution of the API but before sending the response back to the user.
- **AuthorizationManagerAfterMethodInterceptor**, is the one which intercept the @PostAuthorize annotation.

Let's see with an example

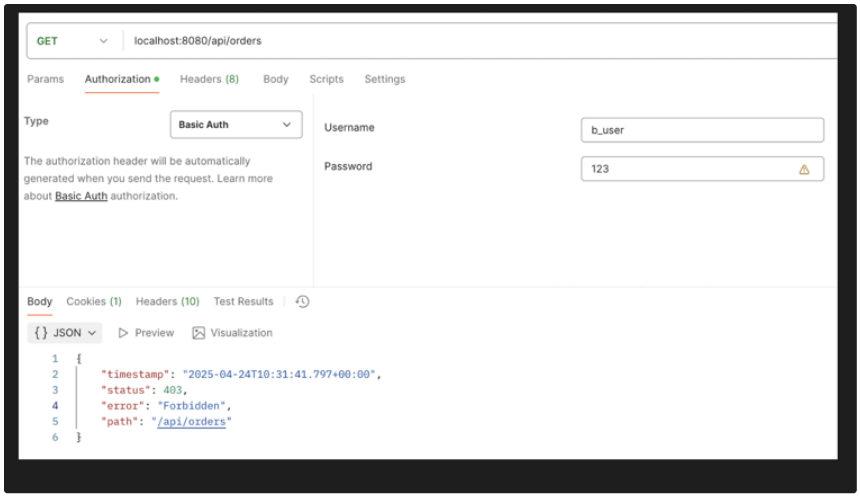
Created 2 Users



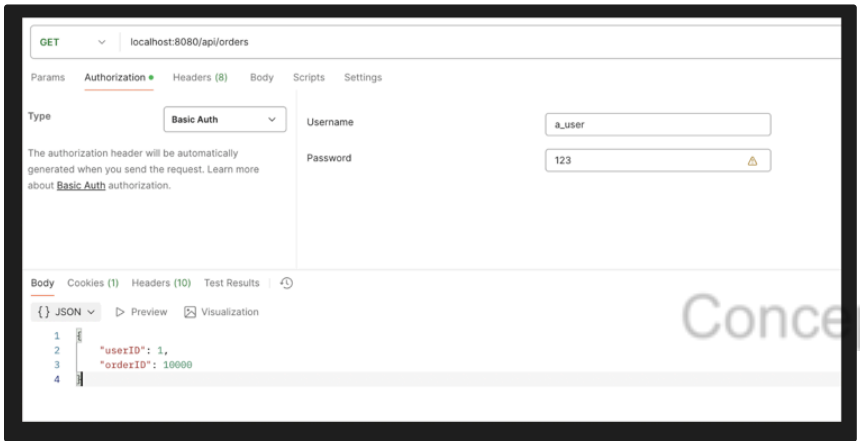
```java
@RestController
@RequestMapping("/api")
public class OrderController {

    @GetMapping("/orders")
    @PreAuthorize("hasRole('USER') and hasAuthority('ORDER_READ')")
    @PostAuthorize("returnObject.userID == authentication.principal.id")
    public OrderDTO readOrders() {

        OrderDTO orderDTO = new OrderDTO();
      /*
      For testing purpose, I have hardcoded the user ID, and added
      the userID of "a_user"
       */
        orderDTO.userID = 11; //hardcoding for now
        orderDTO.orderID = 100001;

        return orderDTO;
    }
}

```

Concept && Coding

Now, try to invoke this api with "b_users" credentials



Now, try to invoke this api with "a_users" credentials



Concept && Coding